# ps1

AUTHOR
YUE WANG

PUBLISHED
October 5, 2024

1. **PS1:** Due Sat Oct 5 at 5:00PM Central. Worth 50 points.

We use (`*`) to indicate a problem that we think might be time consuming.

Steps to submit (5 points on PS1)

1. "This submission is my work alone and complies with the 30538 integrity policy." Add your initials to indicate your agreement: YUE WANG
2. "I have uploaded the names of anyone I worked with on the problem set here" YUE WANG (1 point)
3. Late coins used this pset: **1** Late coins left after submission: **3**
4. Knit your ps1.qmd to make ps1.pdf. The PDF should not be more than 25 pages. Use head() and re-size figures when appropriate.
5. Push ps1.qmd and ps1.pdf to your github repo. It is fine to use Github Desktop.
6. Submit ps1.pdf via Gradescope (4 points)
7. Tag your submission in Gradescope

```
#Set up
import pandas as pd
import warnings
warnings.filterwarnings("ignore", category=pd.errors.DtypeWarning)
```

## Read in one percent sample (15 Points)

## Q1

```
import os
import time

#starting recording the time
starting_time = time.time()

file_path = r"C:/Users/freya/Desktop/yuewang/python2/parking_tickets_one_percent.csv"
file_sample = pd.read_csv(file_path)

# record the ending time
ending_time = time.time()

# calculate the whole execution time
whole_execution_time = ending_time - starting_time

# Assert statement to check the 287458
assert len(file_sample) == 287458
```

```
# print the whole execution time
whole_execution_time
```

```
1.7446057796478271
```

Time taken to read the file follows the results below (unit: seconds).

Successfully add an assert statement which verifies that there are 287458 rows.

# Q2

```
# Get file size in megabytes
file_size= os.path.getsize("C:/Users/freya/Desktop/yuewang/python2/parking_tickets_one_percent

# Predict the full dataset size
full_size = file_size * 100

# Print the results
print(f"1% sample file size: {file_size:.2f} MB")
print(f"Predicted full dataset size: {full_size:.2f} MB")
```

```
1% sample file size: 79.78 MB
Predicted full dataset size: 7978.00 MB
```

The size of the full data set is 7978.00 MB

# Q3

```
# Subset the first 500 rows
file_subset = file_sample.head(500)

# Function
def is_column_ordered(df, file_column_name):
  return df[file_column_name].is_monotonic_increasing

# Write a function that tests if the column is ordered
file_column_name = 'issue_date'
if is_column_ordered(file_subset, file_column_name):
  print(f"The column '{file_column_name}' is ordered in the first 500 cols")
else:
  print(f"The column '{file_column_name}' is not ordered in the first 500 cols")
```

```
The column 'issue_date' is ordered in the first 500 cols
```

## Cleaning the data and benchmarking (15 Points)

# Q1

```
# Load the dataset
df = pd.read_csv("C:/Users/freya/Desktop/yuewang/python2/parking_tickets_one_percent.csv")

# Convert 'issue_date' to datetime
df['issue_date'] = pd.to_datetime(df['issue_date'])
tickets_for_2017 = df[df['issue_date'].dt.year == 2017]

# Count tickets
tickets_for_2017_count = len(tickets_for_2017)
print(f"Tickets in 2017 (sample): {tickets_for_2017_count}")

# Estimate the total number of tickets in the full dataset for 2017
full_data_tickets_for_2017 = tickets_for_2017_count * 100
print(f"Estimated tickets in 2017 (full data): {full_data_tickets_for_2017}")

# ProPublica's reported number of tickets in 2017
propublica_tickets_for_2017 = 3000000

# Calculate the difference
difference = full_data_tickets_for_2017 - propublica_tickets_for_2017
print(f"ProPublica reported tickets: {propublica_tickets_for_2017}")
print(f"Difference: {difference}")
```

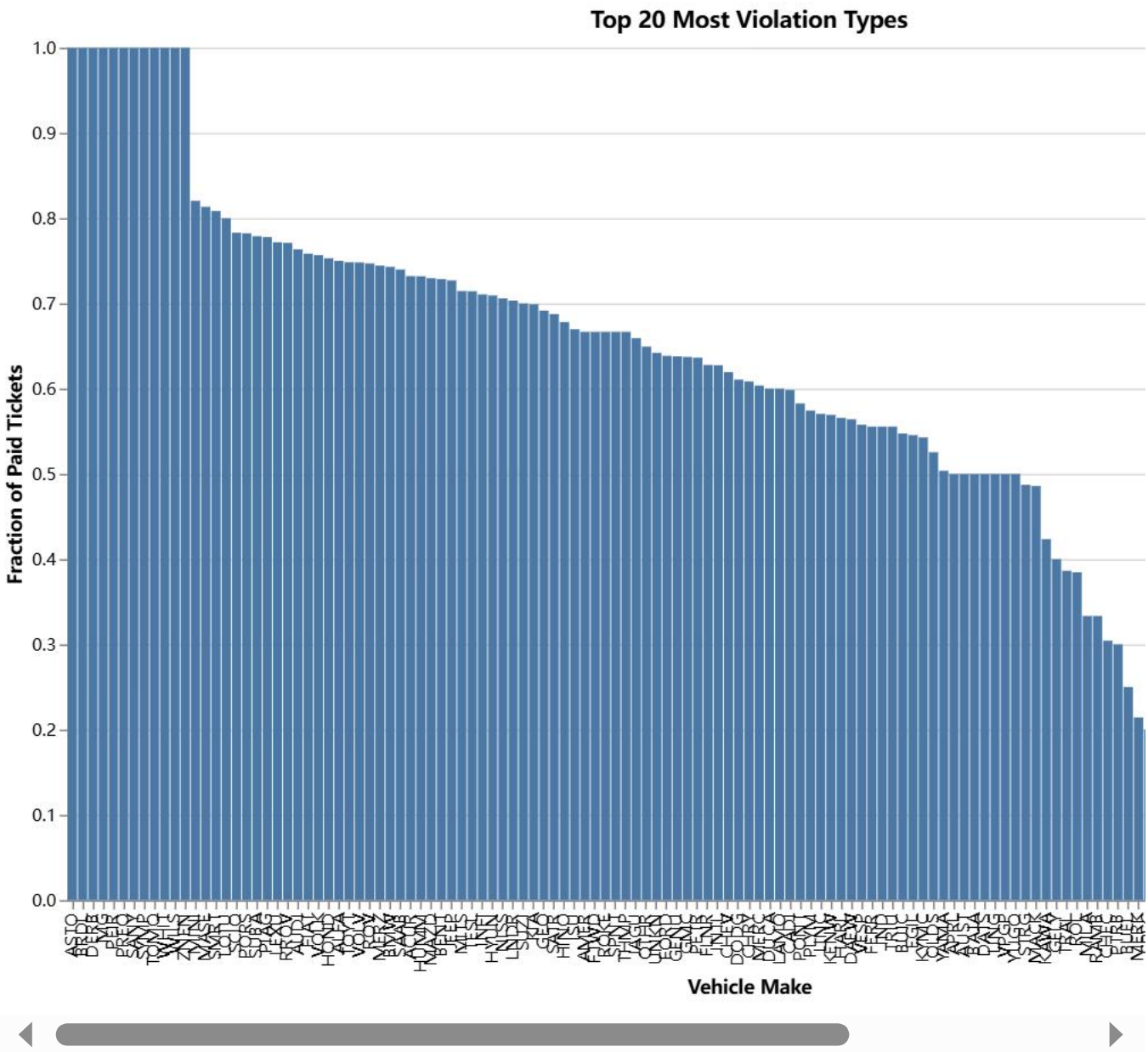
```
Tickets in 2017 (sample): 22364
Estimated tickets in 2017 (full data): 2236400
ProPublica reported tickets: 3000000
Difference: -763600
```

From an academic perspective, the 3 million difference is statistically significant and could introduce bias in future research, affecting conclusions. This discrepancy may result from differences in data scope, such as ProPublica using multi-year averages or variations in data collection and calculation methods, like handling missing or polluted data. It doesn't imply one source is incorrect, but it highlights potential variation. Nevertheless, both figures show that Chicago issues a large number of tickets annually.

# Q2

```
import altair as alt

# Filter out rows with missing data
cleaned_ticket_data = df.dropna(subset=['vehicle_make', 'ticket_queue'])

# Calculate the total number of tickets and the paid tickets
total_tickets_per_make = cleaned_ticket_data.groupby('vehicle_make')['ticket_queue'].count()
paid_tickets_per_make = cleaned_ticket_data[cleaned_ticket_data['ticket_queue'] == 'Paid'].grou

# Calculate the fraction of paid tickets
paid_fraction = (paid_tickets_per_make / total_tickets_per_make).fillna(0).reset_index()
paid_fraction.columns = ['vehicle_make', 'paid_fraction']

# plot the chart
```

```
chart1 = alt.Chart(paid_fraction).mark_bar().encode(
    x=alt.X('vehicle_make:N', title='Vehicle Make', sort='-y'),
    y=alt.Y('paid_fraction:Q', title='Fraction of Paid Tickets'),
    tooltip=['vehicle_make:N', 'paid_fraction:Q']
).properties(
    title='Top 20 Most Violation Types',
    width=800,
    height=500
)

# show the chart
chart1.show()
```




## Visual Encoding (15 Points)

# Q1

| Variable Name | Variable Type(s) |
|---|---|
| `ticket_number` | Nominal |
| `issue_date` | Temporal/Ordinal |
| `violation_location` | Nominal |
| `license_plate_number` | Nominal |
| `license_plate_state` | Nominal |
| `license_plate_type` | Nominal |
| `zipcode` | Nominal/Quantitative |
| `violation_code` | Nominal |
| `violation_description` | Nominal |
| `unit` | Nominal |
| `unit_description` | Nominal |
| `vehicle_make` | Nominal |
| `fine_level1_amount` | Quantitative |
| `fine_level2_amount` | Quantitative |
| `current_amount_due` | Quantitative |
| `total_payments` | Quantitative |
| `ticket_queue` | Nominal |
| `ticket_queue_date` | Temporal |
| `notice_level` | Nominal |
| `hearing_disposition` | Nominal |
| `notice_number` | Nominal |
| `officer` | Nominal |
| `address` | Nominal |

Nominal: Used for categorical data without any inherent order, such as **ticket_number**, **license_plate_number**, and **violation_description**.

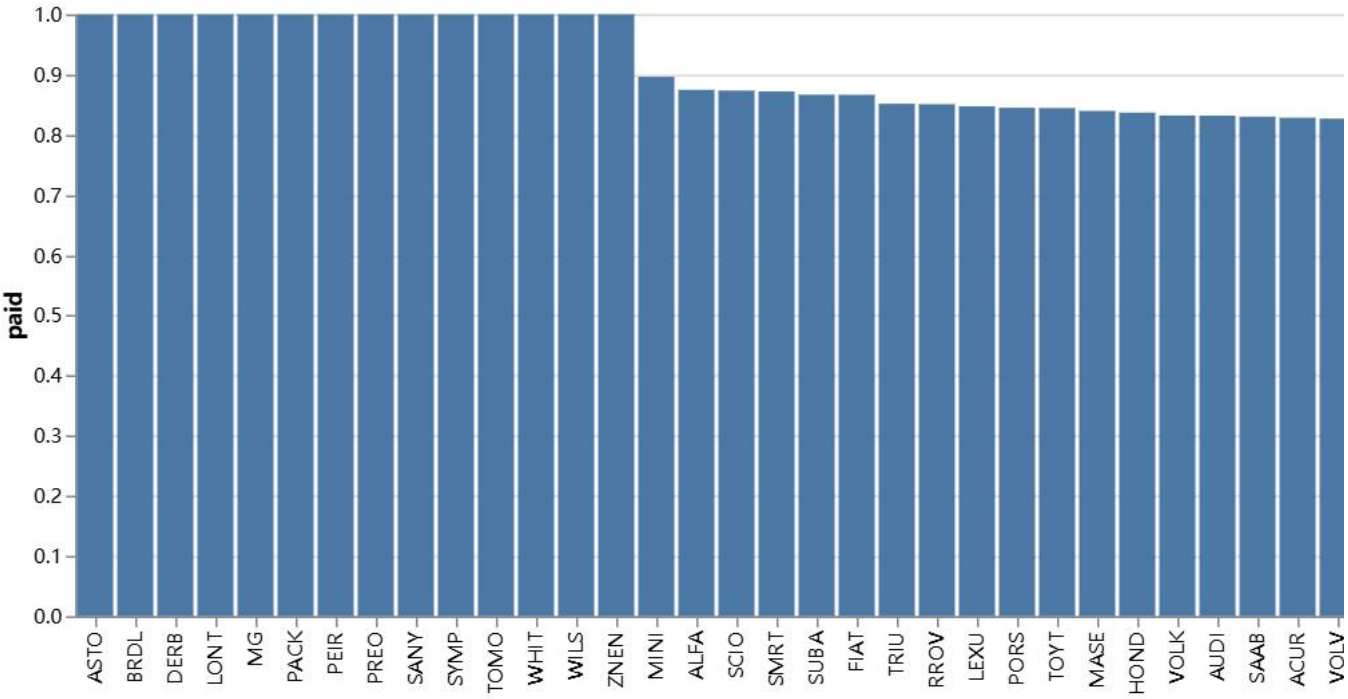
Quantitative: Used for numerical data that can be measured, such as **fine_level1_amount**, **current_amount_due**, and **total_payments**.

Temporal: Used for date or time data, such as **issue_date** and **ticket_queue_date**.

Ordinal: Used when there's a meaningful order, like how **issue_date** could sometimes be used in an ordinal manner depending on how the dataset is analyzed.

# Q2

```
# read the csv file
df = pd.read_csv("C:/Users/freya/Desktop/yuewang/python2/parking_tickets_one_percent.csv", low


# make a column to check if ticket is paid (when 'current_amount_due' is 0)
df['paid'] = df['current_amount_due'] == 0

# get the average paid tickets for each vehicle make
result = df.groupby('vehicle_make')['paid'].mean().reset_index()

# create a bar graph
chart2 = alt.Chart(result).mark_bar().encode(
    alt.X('vehicle_make:N',sort='-y'),
    alt.Y('paid:Q')
).properties(
    title="Paid Tickets by Vehicle Make"
)

chart2.show()
# Save the chart as an HTML file
chart2.save(r'C:/Users/freya/Desktop/yuewang/python2/chart2.html')
```

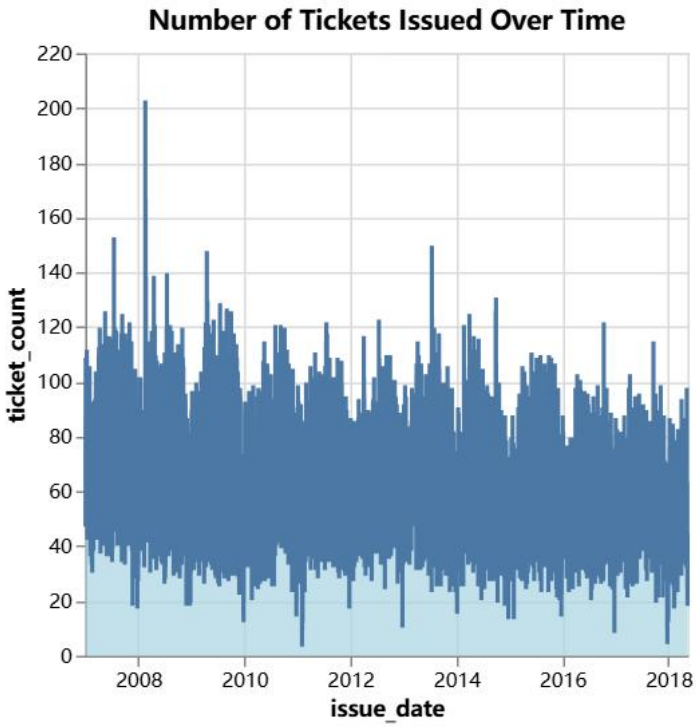


Vehicle owners' financial situations vary significantly by make. Owners of luxury vehicles generally have higher payment rates, as their greater financial resources enable them to pay fines promptly and avoid legal issues. In contrast, owners of mainstream or lower-end brands may face financial challenges, leading to delays or non-payment of tickets.

Additionally, regional differences in law enforcement and policies play a role. Areas with stricter enforcement tend to see quicker fine settlements, further influencing payment behaviors across vehicle makes.

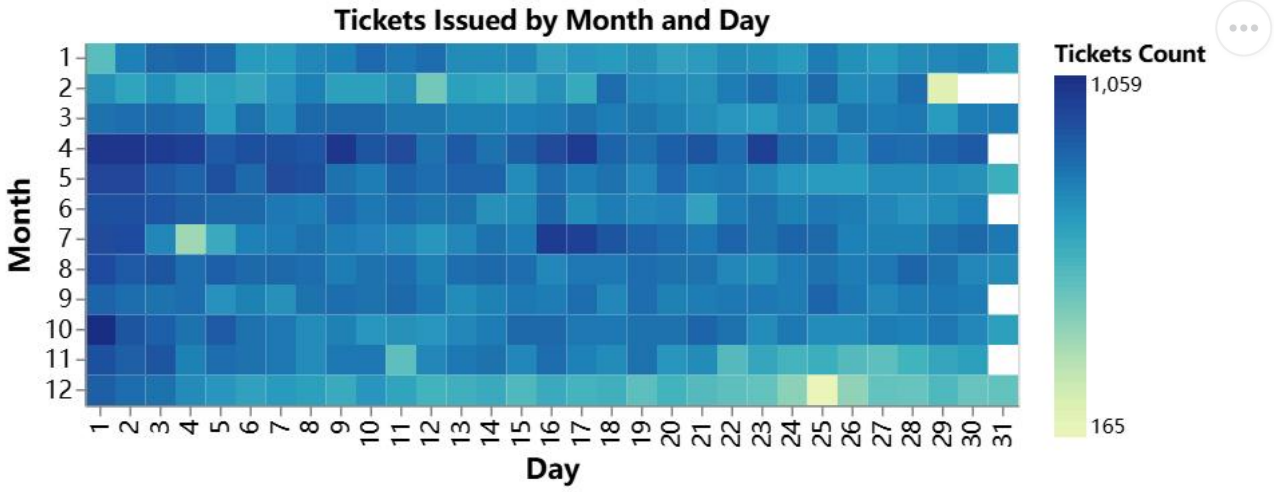
# Q3

```
# Convert 'issue_date' to datetime format
df['issue_date'] = pd.to_datetime(df['issue_date'])

# Count the number of tickets issued per day
tickets_per_day = df.groupby(df['issue_date'].dt.date).size().reset_index(name='ticket_count')

# Rename the date column back to datetime
tickets_per_day['issue_date'] = pd.to_datetime(tickets_per_day['issue_date'])

# Create the chart
chart3 = alt.Chart(tickets_per_day).mark_area(
    color="lightblue",
    interpolate='step-after',
    line=True
).encode(
    x='issue_date:T',  # x-axis is the date
    y='ticket_count:Q'  # y-axis is the number of tickets
).properties(
    title='Number of Tickets Issued Over Time'
)

# Display the chart
chart3.show()
# Save the chart as an HTML file
chart3.save(r'C:/Users/freya/Desktop/yuewang/python2/chart3.html')
```



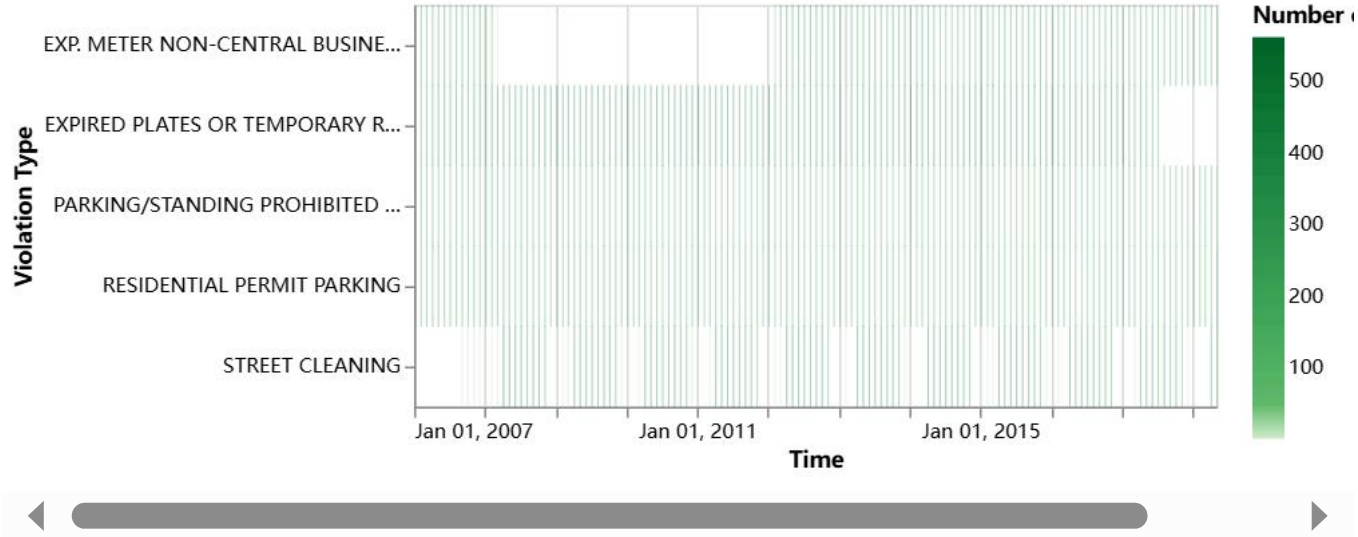

# Q4

```
# Convert 'issue_date' to datetime format
df['issue_date'] = pd.to_datetime(df['issue_date'])

# Extract day and month from the issue date
df['day'] = df['issue_date'].dt.day  # Day of the month
df['month'] = df['issue_date'].dt.month  # Month of the year

# Count the number of tickets issued per day and month
tickets_by_day_month = df.groupby(['month', 'day']).size().reset_index(name='ticket_count')

# Create the graph
heatmap = alt.Chart(tickets_by_day_month, title="Tickets Issued by Month and Day").mark_rect()
    alt.X('day:O', title='Day'),
    alt.Y('month:O', title='Month'),
    alt.Color('ticket_count:Q', title='Tickets Count'),
    tooltip=['month:O', 'day:O', 'ticket_count:Q']
).configure_axis(
    labelFontSize=12,
    titleFontSize=14
).configure_view(
    step=15
)

# Show the graph
heatmap.show()
```




# Q5

```
# Load the dataset
df['issue_date'] = pd.to_datetime(df['issue_date'], errors='coerce')

# Find the five most frequent violation types
common_violations = df['violation_description'].value_counts().nlargest(5).index.tolist()

# Filter the data
filtered_data = df[df['violation_description'].isin(common_violations)]

# Aggregate the number of tickets issued by violation type and date
violation_tickets = filtered_data.groupby(['violation_description', filtered_data['issue_date'

# Convert period back to datetime for visualization
violation_tickets['issue_date'] = violation_tickets['issue_date'].dt.to_timestamp()

# Create the plot
chart4 = alt.Chart(violation_tickets).mark_rect().encode(
    x=alt.X('yearmonthdate(issue_date):T', title='Time'),
    y=alt.Y('violation_description:N', title='Violation Type'),
    color=alt.Color('ticket_count:Q', title='Number of Tickets',
                    scale=alt.Scale(scheme='greens', domainMid=50)),  # Adjust midpoint for da
    tooltip=[
        alt.Tooltip('yearmonthdate(issue_date):T', title='Date'),
        alt.Tooltip('violation_description:N', title='Violation Type'),
        alt.Tooltip('ticket_count:Q', title='Number of Tickets')
    ]
).properties(
    width=400,
    height=200
)



chart4.show()
chart4.save(r'C:/Users/freya/Desktop/yuewang/python2/chart4.html')
```




# Q6

## Filled Step Chart:

### Pros:

1. **Clear Temporal Trends**: The step chart effectively illustrates how the number of tickets fluctuates over time. It shows daily variability and longer-term trends, making it easy to spot peaks and troughs.
2. **Simple to Interpret**: This plot provides a straightforward way to visualize changes, with clear peaks and dips that are easy to identify.

### Cons:

1. **Lack of Detailed Breakdown**: The chart does not provide a breakdown by violation type or other categories, limiting its ability to offer detailed insights.
2. **Limited Granularity**: While it shows overall temporal trends, it does not capture more granular patterns within months or seasons, as it aggregates data by day.

## Annual Weather Heatmap:

### Pros:

1. **Two-Dimensional Representation**: The heatmap effectively shows changes across two dimensions (day and month), allowing viewers to quickly identify trends based on color intensity.
2. **Ideal for Large Datasets**: Heatmaps summarize large datasets well, providing a compact visualization that allows for easy detection of patterns or peaks in the data.

### Cons:

1. **Color Dependence**: Relying heavily on color can be problematic for colorblind users or those who struggle to differentiate between subtle shades.
2. **Loss of Year-to-Year Patterns**: By grouping data by day and month, the heatmap makes it harder to observe year-to-year progression or trends in specific categories.

## Lasagna Plot:

### Pros:

1. **Comparison Across Categories**: This plot allows for simultaneous visualization of trends over time and across different violation categories, providing insights into both dimensions.
2. **Color Intensity Highlights Peaks**: The use of color helps to easily identify periods of high ticket issuance for certain violations, making it simple to compare volumes visually.

### Cons:

1. **Crowded Display Over Long Timeframes**: When the data covers a long period, the plot can become crowded, making it hard to interpret fine details.
2. **Limited Distinction Between Categories**: The color intensity highlights the highest values well but may not clearly distinguish between the second-highest categories or subtle differences across violation types.

# 07

---

## Summarized Answer:

The best plot to illustrate the uneven distribution of violations over time is the **Annual Weather Heatmap**. By using color intensity to represent ticket issuance, the heatmap effectively highlights fluctuations across different days, making it easy to identify peaks and low points in ticket distribution. Unlike the **Filled Step Chart**, which can struggle with large datasets, the heatmap's design is well-suited to convey time-based irregularities without distractions. It simplifies the message by focusing purely on temporal patterns, ensuring the uneven enforcement of violations is the clear takeaway.

However, the **Filled Area Chart** also offers a strong case. Its ability to show distinct peaks and troughs makes it simple for readers to understand the variability in enforcement. As the main focus is on time, the chart presents how ticket issuance changes across different periods, such as year-to-year or month-to-month. Its straightforward structure ensures that readers can quickly recognize how enforcement varies over time, making it another strong contender for illustrating this concept.